TRAFFIC SIGN RECOGNITION

Using Deep Learning and Convolutional Neural Networks (CNN)

Complete Project Roadmap

A beginner-friendly, easy-to-medium project plan for a B.Tech CSE/AIML student, designed for approximately 3–4 months of work.

PROJECT AREA	RECOMMENDED APPROACH
Core task	Classify an image that already contains a traffic sign
Deep learning	CNN built with TensorFlow / Keras
Dataset	German Traffic Sign Recognition Benchmark (GTSRB)
Application	Flask backend + HTML, CSS and JavaScript frontend
Optional extension	Webcam classification; Indian-sign evaluation

Recommended build order: Dataset → Preprocessing → CNN → Training → Evaluation → Prediction → Web app → Testing → Documentation.

1. Project Overview

Project title: Traffic Sign Recognition Using Deep Learning and CNN

Problem statement: Traffic signs communicate important information to drivers. This project develops a deep learning model that classifies a traffic-sign image into a supported category and displays the predicted class and confidence score.

Objectives

- Collect and prepare a labeled traffic-sign dataset.
- Preprocess images and understand the dataset distribution.
- Build and train a CNN using TensorFlow/Keras.
- Evaluate the model using accuracy, precision, recall, F1-score and a confusion matrix.
- Build a simple interface for image upload and prediction.
- Optionally demonstrate classification using a webcam.

Example output

```
Traffic Sign Recognition System
Predicted sign: Speed Limit 50 km/h
Class ID: 2
Confidence: 96.8% (illustrative only; actual results depend on your trained model).
```

Scope note: The first version classifies images that already contain a traffic sign. Finding and locating signs within an entire road scene is a separate, more advanced object-detection task.

2. Recommended Technology Stack

Component	Technology	Purpose
Programming	Python	Main development language
Deep learning	TensorFlow / Keras	Build and train the CNN
Image processing	OpenCV	Image handling and optional webcam frames
Data processing	NumPy, Pandas	Arrays, labels and data inspection
Visualization	Matplotlib, Seaborn	Graphs and evaluation plots
Evaluation	Scikit-learn	Splitting data and metrics
Development	Google Colab / VS Code	Notebook training and application development
Dataset	GTSRB	Labeled traffic-sign images
Web application	Flask + HTML/CSS/JavaScript	Connect model to a user interface
Version control	Git and GitHub	Track code and present the project

Keep the stack simple: Start with Python, TensorFlow/Keras and Google Colab. Add Flask and the frontend only after the model is working.

3. System Architecture

Stage	What happens
1. Input	User uploads an image, or an optional webcam captures one.
2. Preprocessing	Resize image, ensure the expected color format, and normalize pixel values.
3. CNN	Extract visual features and predict probabilities for supported classes.
4. Prediction	Select the class with the highest predicted probability and map it to a sign name.
5. Output	Display the input image, predicted class and confidence score.

Prediction rule: ■ = argmax■ P(y = i | X), where X is the input image and the model selects the class with the highest predicted probability.

4. Dataset Selection

Recommended dataset: GTSRB

The German Traffic Sign Recognition Benchmark (GTSRB) is a common starting point for traffic-sign classification and contains 43 classes. The CS50 traffic-sign project page provides dataset guidance and a useful learning reference:

CS50 Traffic Sign Recognition: https://cs50.harvard.edu/ai/projects/5/traffic/

Tasks

- Download and extract the dataset.
- Inspect its directory structure and class labels.
- Count the images in each class.
- Display sample images and inspect lighting, size and angle variation.
- Plot class distribution and identify possible imbalance.
- Keep the official test set separate if the dataset provides one.

Regional limitation: GTSRB contains German traffic signs. A model trained only on GTSRB should not be assumed to recognize Indian traffic signs reliably. For Indian-sign claims, obtain representative labeled Indian-sign data and evaluate on a separate test set.

5. Complete 16-Week Roadmap

Week	Focus	Tasks	Deliverable
1	Python basics	Revise variables, loops, functions, files, NumPy arrays and Matplotlib. Load and display an image.	Python/image-handling notebook
2	CNN fundamentals	Learn neural networks, convolution, filters, pooling, dense layers, softmax, loss, optimizer, epochs and overfitting.	CNN architecture sketch and notes
3	Dataset exploration	Download GTSRB, inspect labels, display samples, count images and plot class distribution.	Exploration notebook
4	Preprocessing	Resize to 32×32, check RGB/BGR format, normalize pixels, prepare labels and split training/validation data.	Preprocessing pipeline
5	Build CNN	Create a baseline CNN with convolution, pooling, flatten, dense and output layers; compile it.	Compiled CNN
6	Train model	Train for an initial 10–20 epochs, monitor validation metrics and save the best checkpoint.	First trained model
7	Improve model	Try realistic augmentation, dropout, learning-rate changes or a slightly deeper CNN.	At least two recorded experiments
8	Evaluate	Measure accuracy, precision, recall and F1-score; create a confusion matrix and learning curves.	Evaluation notebook
9	Prediction script	Preprocess a single image consistently, predict class probabilities and return class name and confidence.	Working prediction function
10	Flask backend	Load saved model, accept image uploads, validate files, run inference and return results.	Flask prediction endpoint
11	Frontend	Build an upload page, image preview, Predict button and result section with HTML/CSS/JavaScript.	User interface
12	Integration	Connect frontend and backend; test upload, preprocessing, prediction and error handling end to end.	Working web application
13	Optional webcam	Use OpenCV to capture frames and classify a clearly visible sign.	Webcam demo, if time permits
14	Testing	Test clear, blurry, dark, angled and invalid inputs; document failures and fixes.	Test report
15	Documentation	Prepare report, README, setup instructions, architecture diagram and screenshots.	Documented GitHub repository
16	Final presentation	Run from a clean environment, prepare slides, rehearse model explanation and viva questions.	Final submission and demo

6. Image Preprocessing

Apply the same preprocessing during training and inference. Inconsistent preprocessing can cause incorrect predictions.

- **Load:** Read the image from the dataset or upload.
- **Resize:** Use 32 × 32 pixels for the baseline model.
- **Color format:** OpenCV reads BGR by default; convert to RGB if the model expects RGB.
- **Normalize:** Convert pixel values from 0–255 to approximately 0–1 using X_normalized = X / 255.
- **Labels:** Use integer labels with sparse categorical cross-entropy, or one-hot labels with categorical cross-entropy.
- **Split:** A starting split of 80% training and 20% validation is reasonable; use stratification when possible.

• **Test set:** Do not use it to tune the model. Evaluate on it after model development is complete.

7. Baseline CNN Architecture

Layer	Suggested configuration	Purpose
Input	32 × 32 × 3	RGB image input
Conv2D	32 filters, 3 × 3	Extract basic visual features
MaxPooling2D	2 × 2	Reduce feature-map dimensions
Conv2D	64 filters, 3 × 3	Extract more complex features
MaxPooling2D	2 × 2	Further reduce dimensions
Flatten	—	Convert feature maps into a vector
Dense	128 neurons, ReLU	Combine learned features
Dropout	0.3	Help reduce overfitting
Output	43 neurons, softmax	Predict one of 43 classes

Use ReLU in hidden layers and softmax in the output layer. Start with Adam as the optimizer. This is a baseline design; adjust it based on validation results and available computing resources.

8. Training and Model Improvement

Suggested starting settings

• Epochs: 10–20 to begin.

• Batch size: 32.

• Optimizer: Adam.

• Monitor training and validation loss and accuracy.

• Use EarlyStopping, ModelCheckpoint and ReduceLROnPlateau callbacks where appropriate.

Ways to improve

• Apply realistic data augmentation: small rotations, translations, moderate brightness changes and small zoom variations.

• Avoid transformations that change sign meaning. Horizontal flips may reverse directional signs.

• Use dropout or other regularization if overfitting occurs.

• Experiment with a smaller learning rate if validation performance stalls.

• Compare a baseline CNN with a slightly deeper model and record the results.

9. Model Evaluation

Metric	What it tells you
Accuracy	Fraction of all predictions that are correct.
Precision	Among predictions for a class, how many are correct.
Recall	Among actual examples of a class, how many are found.
F1-score	Harmonic mean of precision and recall.
Confusion matrix	Shows which classes are confused with one another.

Create training-vs-validation accuracy and loss plots, a confusion matrix, and a grid of images with predicted and true labels.

Model	Training accuracy	Validation accuracy	Test accuracy
CNN Model 1	Measure in experiment	Measure in experiment	Measure on held-out test set
CNN Model 2	Measure in experiment	Measure in experiment	Measure on held-out test set

Reporting rule: Do not invent or pre-fill performance values. Record the actual results from your own experiments.

10. Single-Image Prediction

Create a prediction function that loads an image, resizes it, converts its color format if needed, normalizes it, passes it to the trained model, and returns the class with the highest probability.

Example probabilities (illustrative): Speed Limit 30 = 0.05, Speed Limit 50 = 0.92, Speed Limit 70 = 0.03. The model selects Speed Limit 50 because it has the highest predicted probability.

A high confidence score does not guarantee correctness. A standard classifier will still choose one of its known classes for an unrelated image unless additional validation is implemented.

11. Flask Application and Suggested Folder Structure

The Flask backend loads the trained model, accepts image uploads, validates files, preprocesses the image, runs inference and returns the class and confidence. The frontend displays the result.

```
traffic-sign-recognition/
■■■ app.py
■■■ train.py
■■■ predict.py
■■■ requirements.txt
■■■ model/traffic_sign_model.keras
■■■ templates/index.html
■■■ static/css/style.css
■■■ static/uploads/ (optional; manage uploads safely)
■■■ notebooks/data_exploration.ipynb
■■■ README.md
```

The dataset may be stored separately if it is too large for GitHub. Avoid committing large datasets or model files unless appropriate; include clear instructions for obtaining them.

Frontend elements

- Project title and short description.
- Choose Image button and image preview.
- Predict Traffic Sign button.
- Result area showing predicted sign, class ID and confidence.
- Optional information about supported classes and evaluation metrics.

End-to-end flow

User uploads image → frontend sends image to Flask → backend preprocesses it → CNN predicts class → backend returns result → frontend displays prediction.

12. Optional Webcam Feature

- Use OpenCV to open the webcam and capture frames.

- Preprocess each frame in the same way as training images.
- Run classification and display the predicted class and confidence.
- For a beginner demo, hold a sign close to the camera or show a clear sign image.

Classifying each full webcam frame does not automatically locate traffic signs within a road scene. Full-scene detection requires a separate object-detection component and should be treated as an advanced extension.

13. Testing and Error Handling

Test input	What to check
Clear sign image	Prediction appears and output format is correct.
Blurry or dark image	System handles uncertainty or poor quality clearly.
Different angle	Observe whether performance changes.
Invalid file type	Show a helpful error rather than crashing.
Large upload	Enforce an upload size limit.
No traffic sign	Avoid implying that the classifier can reliably reject unrelated images without extra validation.
Regional sign not in training data	Document as an unsupported or out-of-distribution case.

A confidence threshold can be used to flag uncertain predictions, but it cannot by itself reliably determine whether an image contains a traffic sign. Use additional validation or a separate detector if that behavior is required.

14. Documentation and GitHub

Project report chapters

- Introduction and problem statement
- Objectives and scope
- Existing and proposed systems
- Literature review
- Requirements analysis
- System architecture and methodology
- Dataset and preprocessing
- CNN architecture and training
- Implementation and results
- Testing, limitations and future scope
- Conclusion and references

GitHub repository checklist

- Organized source code and requirements file.
- README with overview, setup, training and run instructions.
- Architecture diagram and sample output screenshots.
- Actual evaluation metrics and experiment notes.
- Instructions for downloading the dataset and obtaining model artifacts.

15. Recommended Development Environment

Google Colab

Use Colab for dataset exploration, preprocessing, model training, graph generation and evaluation. GPU access depends on current availability and account limits.

Google Colab: https://colab.research.google.com/

Visual Studio Code

Use VS Code for Python scripts, Flask, HTML/CSS/JavaScript, local testing and Git.

Visual Studio Code: https://code.visualstudio.com/

TensorFlow installation guidance: https://www.tensorflow.org/install

Suggested workflow: Train and evaluate in Colab, save the trained model, then use it in the Flask application.

16. Common Challenges and Solutions

Challenge	Possible solution
Low accuracy	Check preprocessing, labels, class distribution and data quality.
Overfitting	Use appropriate augmentation, regularization and validation monitoring.
Similar signs confused	Inspect the confusion matrix and examples from confused classes.
Slow training	Start with a small CNN and use GPU resources if available.
Web predictions differ from notebook	Make training and inference preprocessing identical.
Fluctuating webcam predictions	Use clearer frames and optionally smooth predictions across frames.
Unrelated image gets a traffic-sign label	Add input validation and investigate out-of-distribution handling.
Poor performance on Indian signs	Evaluate representative Indian-sign data and consider additional training data.

This is an educational prototype, not a safety-critical driving system. High accuracy on GTSRB does not guarantee reliability in real road conditions.

17. Features by Priority

Priority	Features
Essential	Dataset loading, preprocessing, CNN training, classification, confidence display, evaluation, image upload and result display.
Recommended	Training/validation graphs, confusion matrix, top-3 predictions, improved error handling, comparison of two CNNs and responsive UI.
Optional advanced	Live webcam classification, Indian traffic-sign support, full-scene detection, mobile optimization and prediction history.

18. Final Deliverables

Deliverable	Expected result
Dataset exploration	Notebook with sample images and class distribution.
Preprocessing	Consistent pipeline for images and labels.
CNN model	Trained traffic-sign classifier.
Saved model	Model artifact used for inference.
Evaluation	Accuracy, precision, recall, F1-score and confusion matrix.
Prediction script	Predicts a sign from a single image.
Web application	Upload image and display prediction.
Optional webcam module	Live classification demonstration.
Repository and report	Organized code, setup instructions and technical documentation.
Presentation	Slides and a working demonstration.

19. First 7 Days: Action Checklist

Day	Action
1	Set up Google Colab; revise Python basics; import NumPy and Matplotlib.
2	Read and display images; inspect dimensions and color channels.
3	Study CNN basics: convolution, filters and pooling.
4	Read about GTSRB and download the dataset.
5	Inspect folders and labels; display sample traffic signs.
6	Count images per class and plot class distribution.
7	Finish the exploration notebook and write the problem statement, objectives and methodology.

By the end of week one, you should have your environment ready, understand the dataset and know what the system is expected to do.

20. Reference Repositories

Use repositories as learning references. Read their licenses and documentation, understand the implementation, and write your own project code rather than submitting a copied project.

Resource	Why it may help
CS50 Traffic Sign Recognition — project guide	Dataset guidance and a structured learning task.
Akshat-GD — Traffic-Sign-Recognition-using-CNN	A starting reference for CNN classification and notebook workflow.
HaliniMurapala — Sign-Board-Categorization-Traffic-Sign-Recognition	Reference for training, image testing and webcam-related ideas.
DYsop — traffic-sign-recognition	A reference for comparing CNN approaches and evaluation workflows.

Final Recommendation

Build in this order: Dataset → Preprocessing → CNN model → Training → Evaluation → Image prediction → Flask web application → Testing → Documentation.

Keep the first version simple. Focus on a correct, evaluated model before adding advanced features. For the viva, be prepared to explain how the CNN learns, why you chose the architecture, how you evaluated it, and what limitations remain.